ACCESO A DATOS

CASO PRACTICO I UD4

ALUMNO CESUR

25/26

Alejandro Muñoz de la Sierra

PROFESOR

Patricio Santiago Fernández Florez

I N T R O D U C C I O N

El desarrollo de software de empresa enfrenta retos constantes. Los sistemas de almacenamiento viejos no entienden la orientación a objetos de forma nativa. Este documento detalla la resolución del Caso Práctico 1 de la Unidad 4. El trabajo resuelve este problema con la gestión de persistencia en bases de datos orientadas a objetos.

Queremos acercar este desarrollo a un entorno laboral real. Buscamos alinearlo con nuestro Trabajo de Fin de Grado. Situamos el requerimiento técnico en nuestra Asociación de Juegos de Mesa. Diseñamos este módulo para el departamento de compras. El gestor registra las reuniones con representantes de editoriales como Devir, Zacatrus o Maldito Games. Estas citas sirven para negociar reposiciones de stock. El objetivo consiste en añadir un registro interno al cliente. Este registro guarda una colección de datos con las fechas exactas de las visitas.

Este proyecto mejora el trabajo previo con bases de datos relacionales. Las aplicaciones Java sufren un desfase común entre los objetos y las tablas de datos. Encajar objetos en filas y columnas resulta difícil. Es como meter un objeto de tres dimensiones en una hoja plana. Este proceso exige un esfuerzo extra en mapeo, claves foráneas y tablas intermedias.

¿Cómo logramos esta persistencia directa? Usamos ObjectDB para resolver este problema desde la base. Este gestor guarda los objetos directamente desde la memoria RAM al disco duro. El proceso no requiere cambios en la estructura. No usamos un lenguaje propio para hablar con la base de datos. Empleamos Java y el estándar JPA. Este sistema ofrece un marco de trabajo firme y común en la industria.

01
CONTEXTO Y PLAN DEL PROYECTO

Esta memoria detalla la construcción del proyecto en Eclipse. Usamos Maven para gestionar las librerías del software con esta herramienta. Explicamos la configuración inicial y el archivo persistence.xml. Describimos las clases principales del sistema. El programa incluye un gestor de conexiones con patrón Singleton. Añadimos una clase DAO para operaciones CRUD. Una interfaz de consola interactiva ayuda a validar el sistema con el usuario.

El equipo de compras de nuestra Asociación de Juegos de Mesa tiene una meta clara. Registraremos las reuniones presenciales entre nuestros comerciales y los proveedores. Visitamos editoriales como Devir, Maldito Games o Zacatrus de forma habitual. ¿Para qué sirve saber la fecha de la última reunión? Ayuda a pedir más stock y permite conseguir productos exclusivos.

En el código, queremos programar una entidad llamada ClienteEditorial. El sistema debe guardar una lista con las fechas de cada visita. Usaremos un array de datos para almacenar estos tiempos de forma nativa.

Este proyecto requiere el uso de ObjectDB y el estándar JPA. Estas herramientas guardan los objetos directamente en la memoria. Así eliminamos el problema del desfase entre los objetos y las tablas relacionales.

MySQL necesita una tabla para clientes y otra para visitas. Debemos unir las tablas con claves ajenas para ver los datos. ObjectDB elimina estos pasos pesados. El motor guarda el objeto Java con su lista interna en el disco duro. El proceso respeta la estructura del código original.

0 2

CONFIGURACIÓN DEL ENTORNO Y PROBLEMAS (ECLIPSE Y MAVEN)

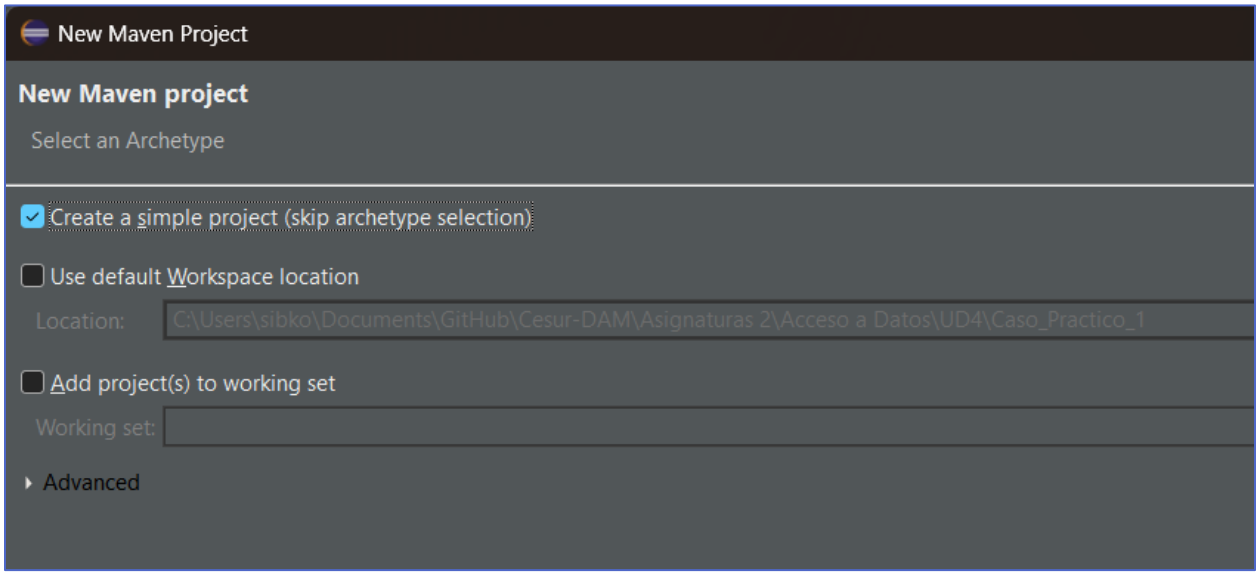
2.1. Creación del proyecto en Eclipse con Maven

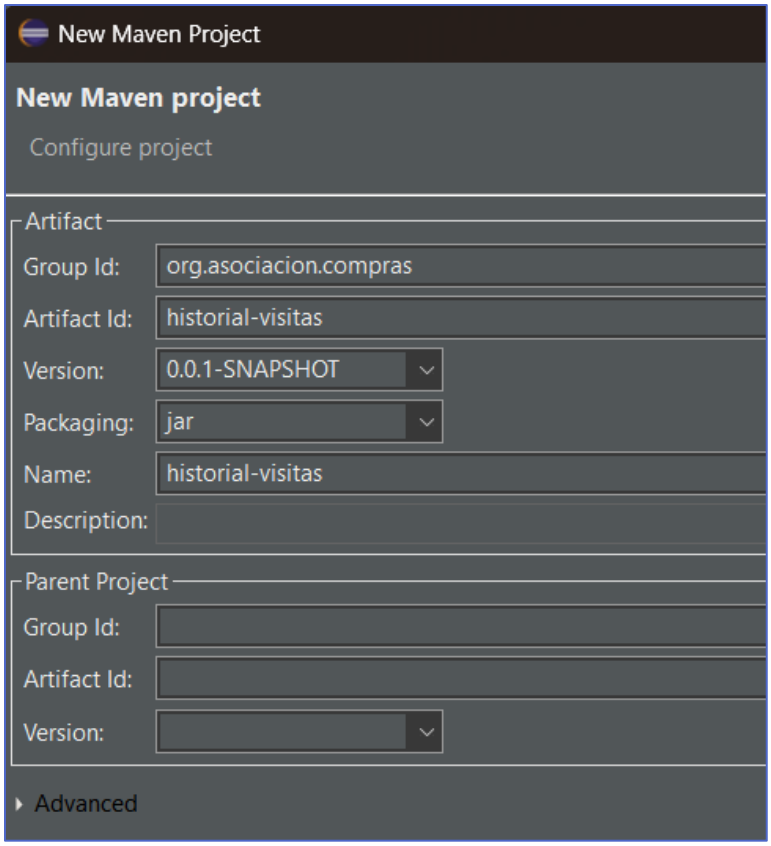
La guía de la práctica pedía crear un proyecto en Eclipse. Nosotros decidimos usar **Maven** por motivos profesionales. Instalar librerías a mano es una tarea antigua. Maven descarga los archivos de forma automática. El programa funciona igual en cualquier equipo.

Abrimos el menú File de Eclipse y seleccionamos New. Después pulsamos en Other y elegimos la opción **Maven Project**.

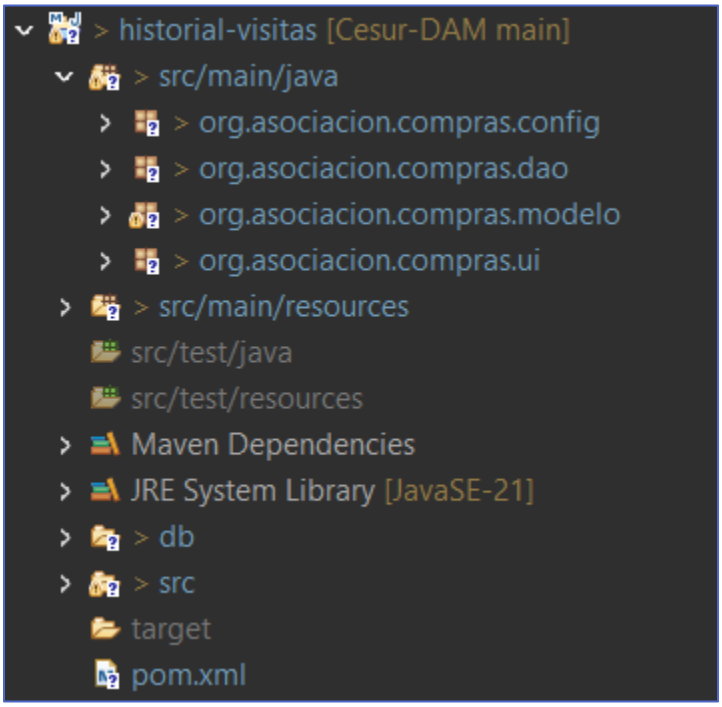
Marcamos la casilla para crear un **proyecto simple sin plantillas**. Queremos el control de todas las carpetas del sistema.

Escribimos **org.asociacion.compras** en el campo **Group Id**. El **nombre** del programa es **historial-visitas**, que coincide con el de **Artifact Id**





Entorno inicial con **JDK 21 LTS**



2.2. Configuración del archivo pom.xml y fallos en las librerías

Tuvimos un **problema** al poner los datos de **ObjectDB** en el archivo **pom.xml**. Eclipse mostró un error en rojo porque **faltaba el archivo .jar**. Leímos los manuales oficiales para buscar la causa. **ObjectDB** es un **producto de pago** con una versión **gratis para alumnos**. Sus archivos **no están en** el sitio normal de **Maven**.

Añadimos la etiqueta de repositorios en el **archivo POM**. Este código indica a Maven la **página** web para **bajar ObjectDB**, además de que **añadir** nuestro **JDK 21** para que no lo machaque con otro por defecto al arrancar. El archivo quedó con este orden:

Acceso a Datos - historial-visitas/pom.xml - Eclipse IDE

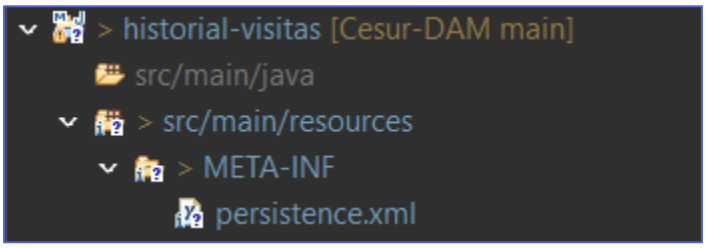
File Edit Navigate Search Project Run Design Window Help

historial-visitas/pom.xml

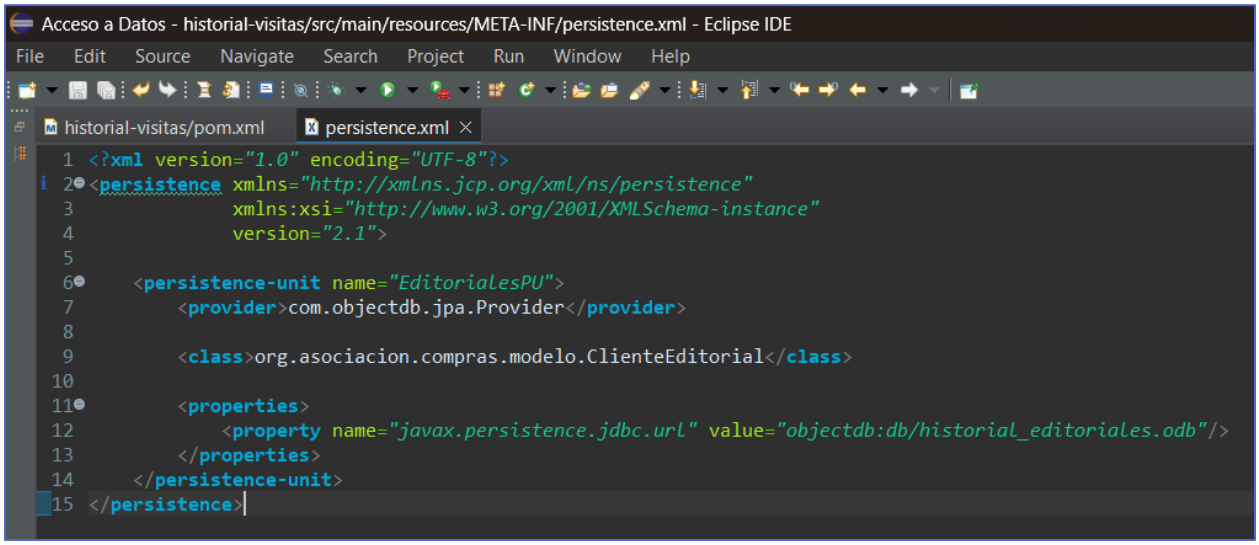
```xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>org.asociacion.compras</groupId>
  <artifactId>historial-visitas</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <properties>
      <maven.compiler.source>21</maven.compiler.source>
      <maven.compiler.target>21</maven.compiler.target>
      <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <repositories>
      <repository>
          <id>objectdb</id>
          <name>ObjectDB Repository</name>
          <url>https://m2.objectdb.com</url>
      </repository>
  </repositories>

  <dependencies>
      <dependency>
          <groupId>com.objectdb</groupId>
          <artifactId>objectdb</artifactId>
          <version>2.8.1</version>
      </dependency>
  </dependencies>
</project>
```

2.3. El archivo persistence.xml

Este archivo es **obligatorio** para que el **estándar JPA** funcione. El sistema lo necesita para saber a qué base de datos ir. También indica qué programa gestiona los datos. Debes crear una **carpeta** llamada **META-IN**F dentro de la ruta **src/main/resources**. Coloca el archivo de configuración **persistence.xml** en esa ubicación. En este lugar definimos nuestra **Unidad de Persistencia**.



Configuramos la **dirección JDBC** hacia un **archivo local** con extensión **.odb**. **ObjectDB crea el archivo** de forma automática al arrancar el programa si no existe todavía.



0 3

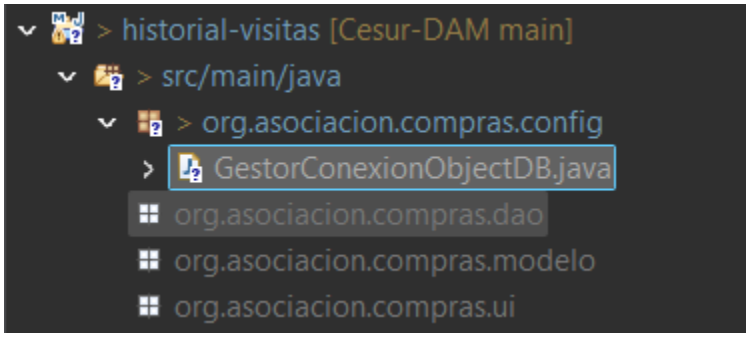
DESARROLLO DE CLASES Y ARQUITECTURA DEL CÓDIGO

Dividimos el sistema en **cuatro grupos**. Esto mantiene el código ordenado. Seguimos el Principio de Responsabilidad Única. Los **paquetes** son **configuración (config), modelo de datos (modelo), acceso a datos (dao) e interfaz de usuario (ui)**.

3.1. Clase de Conexión: GestorConexionObjectDB

Esta **clase** sirve para **gestionar las conexiones** con la base de datos.

El **objeto EntityManagerFactory** consume **muchos recursos en JPA**. Esta herramienta **lee** el archivo **XML** y **analiza** las **clases cuando arranca**. El sistema **prepara los datos** de control y **abre los grupos de conexión**. El programa falla por falta de memoria al crear una factoría nueva para cada usuario. El **patrón Singleton** ayuda al ahorro de memoria y **crea el objeto una sola vez**.



historial-visitas/pom.xml | persistence.xml | GestorConexionObjectDB.java

```java
package org.asociacion.compras.config;

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

/**
 * Clase encargada de gestionar el ciclo de vida de la conexión con ObjectDB.
 * Utilizamos un patrón Singleton para garantizar que la factoría de conexiones
 * se instancie una única vez durante toda la ejecución del programa.
 */
public class GestorConexionObjectDB {

    // Almacenamos la factoría de forma estática.
    private static EntityManagerFactory factoria;

    // Constructor privado para evitar que otros programadores hagan "new
    // GestorConexionObjectDB()"
    private GestorConexionObjectDB() {
    }

    /**
     * Devuelve un manejador de entidades (EntityManager) listo para operar. Si la
     * factoría no existe, la crea leyendo el persistence.xml ("EditorialesPU").
     */
    public static EntityManager obtenerManejador() {
        if (factoria == null || !factoria.isOpen()) {
            // "EditorialesPU" debe coincidir EXACTAMENTE con el <persistence-unit
            // name="..."> del persistence.xml
            factoria = Persistence.createEntityManagerFactory("EditorialesPU");
        }
        // El EntityManager es el objeto que realmente hará los persist(), find(),
        // merge(), etc.
        return factoria.createEntityManager();
    }

    /**
     * Método para liberar recursos al cerrar la aplicación de forma segura.
     */
    public static void cerrarConexion() {
        if (factoria != null && factoria.isOpen()) {
            factoria.close();
        }
```

3.2. Modelo de Datos: ClienteEditorial

Esta **clase** cumple con los **datos requeridos** para el cliente. Incluye el **Id**, el **nombre** y los dos **apellidos**. Añadimos el **comercial** principal, el **Id** de la **empresa** y la **lista** de **visitas**.

Tomamos una decisión técnica sobre la estructura de los datos. El enunciado pide un array. Nosotros usamos una **interfaz List** apoyada por un ArrayList. Los arrays fijos exigen conocer el número de visitas antes de empezar el programa. Este sistema no funciona bien en casos reales. Usamos **List** y la etiqueta **@ElementCollection. JPA y ObjectDB** controlan el **tamaño** de la **lista** según el uso.

historial-visitas/pom.xml | persistence.xml | GestorConexionObjectDB.java | ClienteEditorial.java

```java
1  package org.asociacion.compras.modelo;
2
3  import java.io.Serializable;
4  import java.time.LocalDate;
5  import java.util.ArrayList;
6  import java.util.List;
7
8  import javax.persistence.ElementCollection;
9  import javax.persistence.Entity;
10 import javax.persistence.FetchType;
11 import javax.persistence.GeneratedValue;
12 import javax.persistence.Id;
13
14 /**
15  * Entidad que representa a un representante de una editorial de juegos. Anotada
16  * con @Entity para que ObjectDB sepa que debe hacer persistente esta clase.
17  */
18 @Entity
19 public class ClienteEditorial implements Serializable {
20
21     // Clave primaria autoincremental gestionada por la BDOO
22     @Id
23     @GeneratedValue
24     private Long id;
25
26     private String nombre;
27     private String apellido1;
28     private String apellido2;
29     private String comercialPrincipal;
30     private String idEmpresa; // Identificador interno de la asociación. Ej: "ZACATRUS-001"
31
32     /**
33      * @ElementCollection le dice a JPA que esto no es una relación pura con
34      *                    otra @Entity, sino una colección de tipos básicos
35      *                    (LocalDate). En una BD Relacional crearía una tabla anexa,
36      *                    pero en ObjectDB, sencillamente serializa la lista dentro
37      *                    del mismo objeto de forma nativa. fetch = FetchType.EAGER
38      *                    fuerza a que las fechas se carguen en memoria tan pronto
39      *                    como consultemos al cliente.
40      */
41     @ElementCollection(fetch = FetchType.EAGER)
42     private List<LocalDate> fechasVisitas;
```

```java

    /**
     * Constructor vacío requerido OBLIGATORIAMENTE por la especificación JPA. Lo
     * aprovechamos para inicializar la lista y evitar temidos
     * NullPointerExceptions.
     */
    public ClienteEditorial() {
        this.fechasVisitas = new ArrayList<>();
    }

    /**
     * Constructor sobrecargado para facilitar la creación de objetos desde la
     * consola. No incluimos el 'id' porque de eso se encarga el @GeneratedValue.
     */
    public ClienteEditorial(String nombre, String apellido1, String apellido2, String comercialPrincipal,
            String idEmpresa) {
        this.nombre = nombre;
        this.apellido1 = apellido1;
        this.apellido2 = apellido2;
        this.comercialPrincipal = comercialPrincipal;
        this.idEmpresa = idEmpresa;
        this.fechasVisitas = new ArrayList<>();
    }

    // --- GETTERS Y SETTERS ---
    public Long getId() {
        return id;
    }
    // (Omitimos en este documento los getters/setters de los campos String de texto
    // por brevedad, pero están implementados en Eclipse)

    public List<LocalDate> getFechasVisitas() {
        return fechasVisitas;
    }
```

```java

    /**
     * Método de lógica de negocio encapsulada. En lugar de pedir la lista y hacer
     * un '.add()' desde fuera, le decimos al propio cliente que registre su visita.
     */
    public void registrarVisita(LocalDate fecha) {
        this.fechasVisitas.add(fecha);
    }

    @Override
    public String toString() {
        return "ID: [" + id + "] | Representante: " + nombre + " " + apellido1 + " | Editorial: " + idEmpresa
                + " | Total Visitas Realizadas: " + fechasVisitas.size() + " | Últimas visitas: " + fechasVisitas;
    }
}
```

3.3. Lógica para la base de datos: RepositorioClientesCrud

Este código cumple la regla. Es una **clase** para hacer un **CRUD** con la tabla.

Controlamos el ciclo de **vida** de los **objetos con JPA** aquí. Creamos un **ClienteEditorial** con el comando **new**. El objeto recibe el **estado Transient**. Este estado **no** tiene **conexión** con la base de datos. Usamos **em.persist()** para **cambiar** el objeto **a Managed**. El comando **commit() graba** los datos en el disco. El método **em.merge() actualiza** los registros.

Creamos una función para cada apartado del CRUD

historial-visitas/pom.xml | persistence.xml | GestorConexionObjectDB.java | ClienteEditorial.java | RepositorioClientesCrud.java

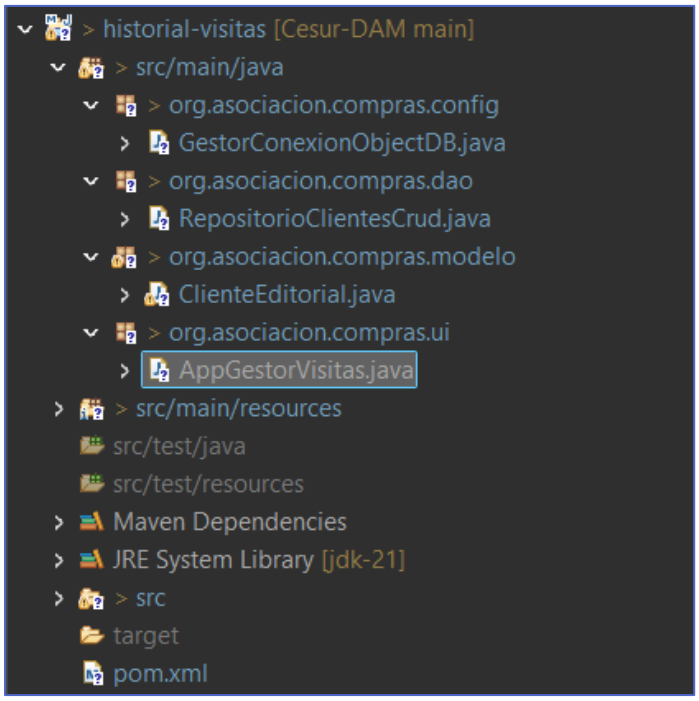
```java
package org.asociacion.compras.dao;

import java.time.LocalDate;
import java.util.List;

import javax.persistence.EntityManager;
import javax.persistence.TypedQuery;

import org.asociacion.compras.config.GestorConexionObjectDB;
import org.asociacion.compras.modelo.ClienteEditorial;

/**
 * Clase DAO (Data Access Object) para concentrar todas las sentencias de base
 * de datos. Aísla el resto de la aplicación de la lógica estricta de JPA.
 */
public class RepositorioClientesCrud {

    // CREATE (Crear)
    public void guardarCliente(ClienteEditorial cliente) {
        EntityManager em = GestorConexionObjectDB.obtenerManejador();
        try {
            em.getTransaction().begin(); // Iniciamos transacción (ACID)
            em.persist(cliente); // Guarda el objeto en la BD
            em.getTransaction().commit();// Confirmamos escritura física
        } catch (Exception e) {
            em.getTransaction().rollback(); // Si algo falla, revertimos para no dejar datos corruptos
            System.err.println("Error crítico al guardar en ObjectDB: " + e.getMessage());
        } finally {
            em.close(); // Siempre liberamos el manejador, incluso si hay error
        }
    }

    // READ (Leer todos)
    public List<ClienteEditorial> listarTodos() {
        EntityManager em = GestorConexionObjectDB.obtenerManejador();
        // Usamos JPQL (Java Persistence Query Language) orientado a objetos, no SQL
        // tradicional.
        // Seleccionamos la clase 'ClienteEditorial', no una supuesta tabla 'clientes'.
        TypedQuery<ClienteEditorial> query = em.createQuery("SELECT c FROM ClienteEditorial c", ClienteEditorial.class);
        List<ClienteEditorial> lista = query.getResultList();
        em.close();
        return lista;
    }
```

```java
    // UPDATE (Actualizar - Añadir la visita al Array/Lista)
    public void añadirVisitaACliente(Long idCliente, LocalDate fechaVisita) {
        EntityManager em = GestorConexionObjectDB.obtenerManejador();
        try {
            em.getTransaction().begin();
            // find() busca el objeto por su Clave Primaria (@Id)
            ClienteEditorial cliente = em.find(ClienteEditorial.class, idCliente);

            if (cliente != null) {
                // Modificamos el objeto en memoria RAM
                cliente.registrarVisita(fechaVisita);
                // merge() empuja los cambios de la RAM al archivo físico .odb de ObjectDB
                em.merge(cliente);
                System.out.println("-> Visita registrada correctamente con fecha: " + fechaVisita);
            } else {
                System.out.println("-> ERROR: No existe ningún representante con el ID: " + idCliente);
            }
            em.getTransaction().commit();
        } finally {
            em.close();
        }
    }

```

```java
    // DELETE (Borrar)
    public void eliminarCliente(Long id) {
        EntityManager em = GestorConexionObjectDB.obtenerManejador();
        try {
            em.getTransaction().begin();
            ClienteEditorial cliente = em.find(ClienteEditorial.class, id);

            if (cliente != null) {
                // remove() elimina el objeto físico de la BDOO
                em.remove(cliente);
                System.out.println("-> Cliente y todo su historial de visitas han sido eliminados.");
            } else {
                System.out.println("-> ERROR: Imposible borrar. ID no encontrado.");
            }
            em.getTransaction().commit();
        } finally {
            em.close();
        }
    }
}
```

3.4. Pruebas y funcionamiento: AppGestorVisitas

Creamos una **interfaz** para usar en la **consola**. **Protegimos** las entradas de datos con bloques **try-catch** para mejorar el funcionamiento. El sistema solicita un código numérico. Si el usuario escribe la palabra "dos" por error, capturamos el fallo NumberFormatException. Esto impide que la aplicación se detenga de golpe.



historial-visitas/pom.xml | persistence.xml | GestorConexionObjectDB.java | ClienteEditorial.java

```java
package org.asociacion.compras.ui;

import java.time.LocalDate;
import java.util.List;
import java.util.Scanner;

import org.asociacion.compras.config.GestorConexionObjectDB;
import org.asociacion.compras.dao.RepositorioClientesCrud;
import org.asociacion.compras.modelo.ClienteEditorial;

/**
 * Clase principal (Main) que arranca la aplicación, muestra el menú y recolecta
 * los datos de entrada del usuario comercial.
 */
public class AppGestorVisitas {

    private static RepositorioClientesCrud crud = new RepositorioClientesCrud();
    private static Scanner scanner = new Scanner(System.in);

    public static void main(String[] args) {
        System.out.println("==================================================");
        System.out.println("=== ASOCIACIÓN JUEGOS: HISTÓRICO VISITAS B2B ===");
        System.out.println("==================================================");

```

```java
        boolean salir = false;

        while (!salir) {
            mostrarMenu();
            try {
                // Usamos nextLine() y parseamos para evitar el bug del salto de línea del
                // Scanner
                int opcion = Integer.parseInt(scanner.nextLine());

                switch (opcion) {
                case 1:
                    crearNuevoCliente();
                    break;
                case 2:
                    listarClientes();
                    break;
                case 3:
                    registrarVisitaInteractiva();
                    break;
                case 4:
                    borrarClienteInteractivo();
                    break;
                case 5:
                    salir = true;
                    // Importantísimo cerrar la factoría de JPA antes de apagar la JVM
                    GestorConexionObjectDB.cerrarConexion();
                    System.out.println("Cerrando la base de datos de ObjectDB. ¡Hasta pronto!");
                    break;
                default:
                    System.out.println("-> Opción no válida. Introduce un número del 1 al 5.");
                }
            } catch (NumberFormatException e) {
                System.out.println("-> ERROR: Por favor, introduce un número válido, no texto.");
            }
        }
    }
```

```java
    private static void mostrarMenu() {
        System.out.println("\n--- MENÚ PRINCIPAL ---");
        System.out.println("1. Alta de nuevo representante (Cliente)");
        System.out.println("2. Ver listado completo y total de visitas");
        System.out.println("3. Añadir fecha de visita a un representante");
        System.out.println("4. Dar de baja a un representante");
        System.out.println("5. Guardar y Salir");
        System.out.print("Elige una acción >> ");
    }

    private static void crearNuevoCliente() {
        System.out.println("\n-- NUEVO REGISTRO --");
        System.out.print("Nombre: ");
        String nombre = scanner.nextLine();
        System.out.print("Primer Apellido: ");
        String ape1 = scanner.nextLine();
        System.out.print("Segundo Apellido: ");
        String ape2 = scanner.nextLine();
        System.out.print("Tu nombre (Comercial Principal): ");
        String comercial = scanner.nextLine();
        System.out.print("ID de la Editorial (Ej: DEVIR-01): ");
        String idEmpresa = scanner.nextLine();

        // Instanciamos el objeto Transient
        ClienteEditorial nuevo = new ClienteEditorial(nombre, ape1, ape2, comercial, idEmpresa);
        // Delegamos en el DAO la persistencia
        crud.guardarCliente(nuevo);
        System.out.println("-> Representante " + nombre + " añadido a la base de datos.");
    }
```

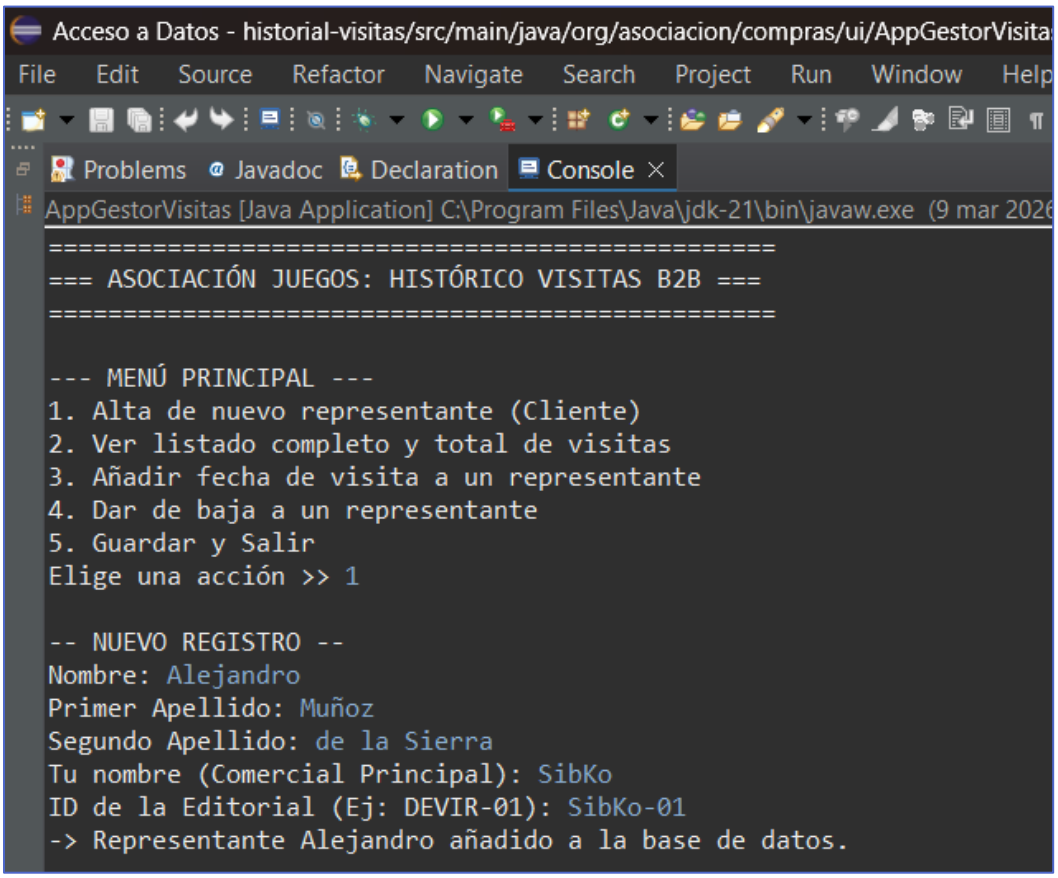
```java

    private static void listarClientes() {
        System.out.println("\n-- LISTADO DE REPRESENTANTES Y VISITAS --");
        List<ClienteEditorial> lista = crud.listarTodos();
        if (lista.isEmpty()) {
            System.out.println("La base de datos está vacía.");
            return;
        }
        for (ClienteEditorial c : lista) {
            // Se invoca automáticamente el método toString() sobrescrito en el modelo
            System.out.println(c);
        }
    }

    private static void registrarVisitaInteractiva() {
        System.out.println("\n-- REGISTRAR NUEVA VISITA --");
        listarClientes(); // Mostramos los clientes para que el usuario vea los IDs
        System.out.print("\nIntroduce el ID numérico del representante que has visitado: ");
        try {
            Long id = Long.parseLong(scanner.nextLine());
            // Registramos la visita con la fecha del sistema en este instante
            crud.añadirVisitaACliente(id, LocalDate.now());
        } catch (NumberFormatException e) {
            System.out.println("-> ERROR: El ID debe ser un número entero.");
        }
    }

    private static void borrarClienteInteractivo() {
        System.out.println("\n-- DAR DE BAJA --");
        System.out.print("Introduce el ID del representante a eliminar: ");
        try {
            Long id = Long.parseLong(scanner.nextLine());
            crud.eliminarCliente(id);
        } catch (NumberFormatException e) {
            System.out.println("-> ERROR: El ID debe ser un número entero.");
        }
    }
}
```

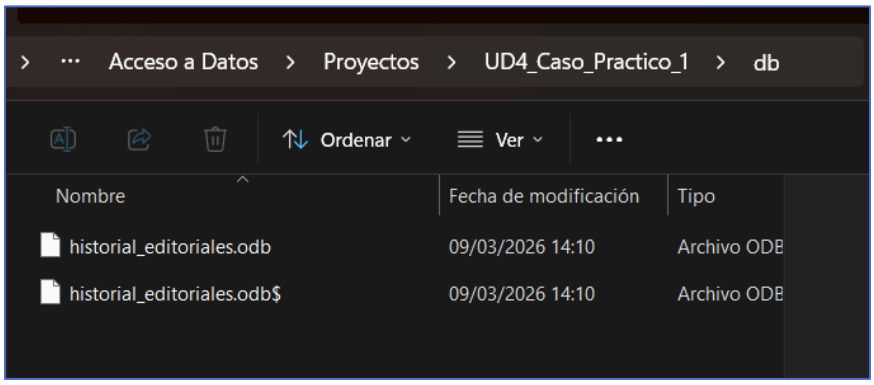
0 4

E J E C U C I O N Y P R U E B A S

Iniciamos el programa en nuestro entorno de desarrollo. **ObjectDB crea** el archivo físico llamado **historial_editoriales.odb** por su cuenta. El archivo aparece dentro de la **carpeta db**. Esto ocurre siguiendo las **indicaciones** del archivo **persistence.xml**.

Comprobamos el funcionamiento de todo el **ciclo de datos**. La función de **crear** guarda a los representantes con un ID automático que crece solo.





```
--- MENÚ PRINCIPAL ---
1. Alta de nuevo representante (Cliente)
2. Ver listado completo y total de visitas
3. Añadir fecha de visita a un representante
4. Dar de baja a un representante
5. Guardar y Salir
Elige una acción >> 2

-- LISTADO DE REPRESENTANTES Y VISITAS --
ID: [1] | Representante: Alejandro Muñoz | Editorial: SibKo-01 | Total Visitas Realizadas: 0 | Últimas visitas: []
```

La **actualización** añade fechas a la lista de visitas de cada cliente sin fallos. La consola muestra el grupo de fechas con un formato limpio

```
--- MENÚ PRINCIPAL ---
1. Alta de nuevo representante (Cliente)
2. Ver listado completo y total de visitas
3. Añadir fecha de visita a un representante
4. Dar de baja a un representante
5. Guardar y Salir
Elige una acción >> 3

-- REGISTRAR NUEVA VISITA --

-- LISTADO DE REPRESENTANTES Y VISITAS --
ID: [1] | Representante: Alejandro Muñoz | Editorial: SibKo-01 | Total Visitas Realizadas: 0 | Últimas visitas: []

Introduce el ID numérico del representante que has visitado: 1
-> Visita registrada correctamente con fecha: Mon Mar 09 15:50:01 CET 2026
```

El **borrado** elimina al cliente y sus fechas guardadas de una vez. El sistema no deja datos perdidos que ocupen espacio en la memoria.

```
--- MENÚ PRINCIPAL ---
1. Alta de nuevo representante (Cliente)
2. Ver listado completo y total de visitas
3. Añadir fecha de visita a un representante
4. Dar de baja a un representante
5. Guardar y Salir
Elige una acción >> 4

-- DAR DE BAJA --
Introduce el ID del representante a eliminar: 1
-> Cliente y todo su historial de visitas han sido eliminados.
```

Resultado:

Esta práctica enseña la fuerza de una Base de Datos Orientada a Objetos. Este sistema es más útil que las bases de datos de tablas en muchos casos. Gestionamos los datos con el estándar JPA de forma sencilla. No diseñamos tablas ni usamos llaves ajenas. No escribimos frases complejas al guardar una lista de fechas dentro de un objeto. ObjectDB guarda el estado del objeto como está en la memoria RAM. Este método acelera mucho la creación de programas.

0 5
C O N C L U S I O N E S

Construimos una aplicación en Java que gestiona información en una base de datos orientada a objetos. Establecimos conexiones seguras y un CRUD interactivo. Los datos mantienen su presencia física en el disco.

El mayor aprendizaje nació al escribir el código y enfrentar errores de tipos. Modelamos el registro de fechas con la API de Java 8 al principio. Usamos java.time.LocalDate como recomienda el lenguaje moderno. El código funcionaba bien en el editor. El programa falló con una RollbackException durante el guardado del objeto en la base de datos.

Descubrimos el motivo del error. Existe un conflicto entre JPA 2.1 y la API de Java 8. ObjectDB 2.8 incluye JPA 2.1 de serie. Esta versión no guarda un LocalDate sin un AttributeConverter manual. JPA 2.2 incluyó esta función más tarde. Tomamos una decisión práctica. Refactorizamos la colección para usar la clase clásica java.util.Date.

Comprobamos la libertad que dan las BDOO frente a MySQL con datos reales. Guardamos una lista de fechas en un objeto ClienteEditorial. Solo usamos la anotación @ElementCollection. No diseñamos tablas intermedias como cliente_visitas. No creamos claves foráneas. No escribimos consultas JOIN complejas para leer el historial.

El sistema final cumple los requisitos pedidos sobradamente. Entregamos un código ordenado en capas lógicas. La interfaz de consola perdona los errores del usuario. La arquitectura está lista para el futuro y permite el crecimiento.

0 6
R E F E R E N C I A S

https://www.objectdb.com/java/jpa

https://www.objectdb.com/tutorial/jpa/start/maven

https://www.objectdb.com/tutorial/jpa/eclipse

https://docs.oracle.com/javaee/7/tutorial/partpersist.htm

https://stackoverflow.com/questions/37166291/jpa-2-1-introducing-java-8-date-time-api

https://thorben-janssen.com/persist-localdate-localdatetime-jpa/

Cómo crear y configurar un proyecto Maven en Eclipse
https://www.youtube.com/watch?v=L_U3uYXyTRU

@Entity con JPA en JAVA - Tutorial Completo Fácil
https://www.youtube.com/watch?v=skd4Qq5DT9Y

Como usar las Anotaciones @Entity, @Table, @Column, @ID en Java con JPA | Curso Java Swing JPA
https://www.youtube.com/watch?v=JbSivbPHzwo

Diseño de una Base de Datos Orientada a Objetos BDOO
https://www.youtube.com/watch?v=JJFlNnEXM68